Chapter 6

Conversational Analytics and Agentic Workflows with Gemini in Looker

6.1 Introduction and Business Motivation

The deployment of centralized data warehouses and semantic modeling frameworks has historically resolved technical consistency, relational integrity, and single-source metric definition; however, an acute operational bottleneck frequently persists at the point of data consumption. In vertically integrated agro-industrial supply chains such as Amadori S.p.A., operational planners, procurement managers, and plant supervisors require rapid, ad-hoc access to tactical information to adjust to biological variances and volatile retail market shifts.

Traditionally, extracting answers from enterprise data repositories mandates either high Business Intelligence proficiency—such as manually constructing complex queries across multidimensional Looker Explores—or submitting ticketing requests to centralized data engineering teams. This structural friction introduces operational latency, encourages the proliferation of unmanaged shadow spreadsheets, and widens the gap between transactional data capture and executive decision-making.

To bridge this interaction boundary, the target enterprise architecture implements Google Cloud's Generative AI framework through **Gemini in Looker**. Rather than treating artificial intelligence as an unconstrained Text-to-SQL code generator operating against raw relational schemas—a pattern inherently vulnerable to relational hallucinations, uncalibrated aggregations, and Cartesian join fanouts—this implementation anchors conversational interactions strictly to the certified **Semantic Layer** formalized in Section 5.2.

By restricting the large language model (LLM) to interact exclusively through curated LookML models, views, and explores, the system guarantees that every analytical insight generated remains mathematically verified, aligned with certified enterprise logic, and governed by centralized access controls. This chapter formalizes the operationalization of this conversational paradigm: it provides an architectural taxonomy of Looker's conversational modalities, details the iterative engineering methodology driven by diagnostic reasoning logs (*Show Thinking*), illustrates the progressive migration of business logic toward the core backend, defines the governance of multi-scale visualizations, and establishes a formal validation suite centered on enterprise **Golden Queries**.

6.2 Architectural Framework: Conversational Modalities in Looker

Google Cloud structures natural language analytical interactions within Looker across three tiered, specialized architectural modalities [11]:

1. **Chat with Dashboard:** An analytical copilot tightly coupled to a specific, pre-rendered dashboard. It operates exclusively upon pre-aggregated data rendered within existing visualization tiles, delivering qualitative summaries, identifying visual outliers, and synthesizing cross-tile comparisons without generating or executing underlying SQL queries against the analytical database.

2. **Chat with Explore:** A contextual assistant embedded directly within a single Looker Explore interface. Designed primarily for self-service analysts, it assists in ad-hoc query synthesis by translating natural language prompts into valid dimensions, measures, and filter clauses constrained strictly to that Explore's perimeter.

3. **Conversational Data Agents (Chat with Your Data):** The enterprise-grade agentic architecture selected for this thesis project. Provisioned through the Conversational Analytics framework, this modality instantiates an autonomous agent powered by Gemini capable of orchestrating multi-turn dialogues, resolving domain ambiguities, applying contextual heuristics, and compiling structured LookML queries across certified Explores.

In the Amadori Material Planning implementation, the Conversational Data Agent is bound directly to the `material_planning` explore. The agent acts as an interactive operational copilot, translating complex planning inquiries into governed semantic queries and determining whether to return synthesized textual

responses, native Looker visual widgets, or specialized Python-driven statistical charts.

6.3 System Instructions: Codifying Enterprise Domain Knowledge

Large Language Models possess general linguistic competence but lack intrinsic awareness of specialized corporate taxonomy, production flows, and manufacturing constraints. To bridge this semantic gap, the agent's cognitive behavior is governed by a structured set of **System Instructions** that define its operational persona, inject core domain taxonomy, and enforce strict execution guardrails.

6.3.1 Persona and Proactive Dialogue Policy

The agent is instantiated as an authoritative, professional data specialist dedicated to Amadori's Material Planning unit. Its behavioral policy enforces two core interaction tenets:

- **Direct, Data-Grounded Responses:** The agent delivers synthesized, fact-based answers immediately, eliminating conversational preambles, speculative commentary, and generic conversational filler.
- **Proactive Analytical Steering:** Operational planning requires dynamic problem-solving. At the conclusion of every response, the agent is mandated to anticipate planner needs by proposing 2–3 relevant follow-up questions or tactical next steps (e.g., suggesting an inspection of packaging safety stock after an ingredient requirement spike is detected).

6.3.2 Codification of Agri-Food Supply Chain Taxonomies

Material planning in the poultry processing industry depends on multi-stage conversion cycles. The system instructions explicitly formalize Amadori's production stages and item relationships:

- **Gross Requirement (*Fabbisogno Lordo*):** Total raw material or packaging volume calculated from downstream finished goods forecasts, mapped deterministically to the core additive measure `f_material_planning.fabbisogno_materiale`.
- **On-Line Finished Product (PFL - *Prodotto Finito in Linea*):** Finished commercial goods assembled and packaged in a continuous on-line manufacturing process (combining ingredients, primary packaging, and labels).

- **Neutral Product (PN - *Prodotto Neutro*):** Base semi-finished products that are processed and packed in neutral format without customer-specific labels, referenced through the master ERP field `l_art_jde.articolo_neutro_cod`.
- **Finished Product from Neutral (PFN - *Prodotto Finito da Neutro*):** Commercial SKUs finalized downstream by affixing customer-specific labels to a parent PN.
- **Packaging Labels (*Etichette*):** Essential tracking components identified via pattern matching where `l_art_jde.materiale_des` contains `"Et"` or `"Etich"`.
- **Lexical Equivalence:** Explicitly binding colloquial operational terminology, mandating that *"vaschetta"* and *"vassoio"* (thermoformed packaging trays) are treated as identical synonyms within descriptive filters.

6.3.3 Operational Defaults and Execution Guardrails

To prevent database query bloat, unconstrained full-table scans, and relational misinterpretations, the instructions hardcode mandatory operational boundaries:

- **Measure Routing:** Ambiguous user requests regarding "quantities", "requirements", or "demand" default deterministically to `fabbisogno_materiale`. Queries targeting physical finished product units route to `f_material_planning.casse_art`
- **Mandatory Platform Perimeter:** Whenever an inquiry references logistics deposits or manufacturing plants generically—or omits platform criteria altogether—the agent is mandated to apply a strict set filter restricting execution to Amadori's certified production hubs:

$$\texttt{f_material_planning.piattaforma_cod} \in \{\text{S01, S02, S04, S05, S10, S50}\} \tag{6.1}$$

- **Temporal Fallbacks:** Open-ended operational inquiries default to a window spanning from the current calendar date to the end of the subsequent calendar month. For long-term strategic inquiries, the temporal fallback automatically bounds execution to the next 12 rolling months.
- **Mandatory Metric-Unit Pairing:** To prevent invalid dimensional interpretations, the agent is prohibited from displaying raw values of `fabbisogno_materiale` in isolation. Every numeric output must be paired with its native Unit of Measure (`f_material_planning.um_cod`), explicitly decoded for business clarity: `MQ` (square meters), `NR` (pieces/units), `KG` (kilograms), or `MT` (linear meters).

6.4 Iterative Agent Engineering and Logic Migration: Pushing Transformations Upstream

Developing an enterprise-grade agent requires an empirical tuning loop. The Material Planning agent was optimized using Looker's native **Show Thinking** diagnostic interface, which exposes the LLM's step-by-step reasoning trace, token parsing logic, filter assumptions, and tool invocation choices prior to query execution.

6.4.1 The Architectural Journey of Business Logic: The Three-Stage Maturation Cycle

As established in the architectural best practices in Section 5.2, high-performance semantic layers adhere to the core tenet of **pushing transformations upstream**. Complex data classifications, recursive allocations, and heavy row-level operations should reside in upstream data pipelines or database layers rather than being computed dynamically at consumption time.

The operationalization of the material classification logic (`tipo_materiale`, separating ingredients from packaging) provides a clear case study of this architectural migration, progressing through three distinct developmental stages:

1. **Stage 1: Generative LLM Heuristics (Runtime Code Interpretation).** Initial natural language queries demanding material segregation (e.g., *"Dividi gli ingredienti dagli imballi"*) caused the agent to pause. The *Show Thinking* trace revealed that Gemini invoked its integrated Python Code Interpreter to run inductive string-pattern heuristics across thousands of retrieved raw ERP codes:

   ```
   "Codes starting with '0' seem to represent ingredients (Budello,
   Olio), while codes starting with 'I' represent packaging (Film,
   Vassoio)...  Retrieving raw codes to run python_agent categorization..."
   ```

   While analytically correct, executing statistical pattern matching across thousands of returned rows introduced massive operational latency (exceeding 25 seconds), exhausted LLM context tokens, and generated severe non-deterministic variance.

2. **Stage 2: Semantic Layer Formalization (Compiled LookML Dimension).** To eliminate runtime Python execution, the classification logic was formalized inside the LookML master view as a compiled dimension using a SQL `CASE` statement:

```
dimension: tipo_materiale {
  type: string
  label: "Tipo Materiale"
  sql: case
         when ${materiale_lcod} like 'I%' then '
            Imballo'
         when ${materiale_lcod} like '0%' then '
            Ingrediente'
         else 'Altro'
       end ;;
  synonyms: ["categoria materiale", "imballo vs
     ingrediente"]
}
```

This shift successfully eliminated the Python code interpreter invocation, forcing Looker to compile deterministic `GROUP BY` queries directly in SQL. However, evaluating row-level string comparisons dynamically across large-scale fact tables during peak reporting cycles introduced relational database overhead.

3. **Stage 3: Enterprise Backend Consolidation (Target ELT Architecture).** In alignment with Looker's performance best practice of offloading row-level calculations from the semantic layer, the classification rule was permanently migrated into the upstream batch ELT pipeline. The physical master table (`L_ART_JDE`) was structurally enriched with the native, indexed attribute `TIPO_MAT`. Consequently, the LookML declaration was simplified to a direct physical column mapping:

```
dimension: tipo_materiale {
  type: string
  label: "Tipo Materiale"
  sql: ${TABLE}.TIPO_MAT ;;
  description: "Classificazione del materiale tra
     imballaggio e ingrediente primario"
  suggestions: ["Imballo", "Ingrediente"]
}
```

This evolutionary trajectory encapsulates a fundamental insight for Digital Transformation Management: *Large Language Models and Semantic Layers should not be utilized as computational calculation engines for deterministic business*

rules. Instead, operational heuristics discovered during conversational interactions must be systematically captured, codified in the semantic layer, and ultimately pushed upstream into the enterprise data architecture.

6.4.2 Resolution of Edge Cases: Exact Matches and Superset Traps

Inspection of diagnostic reasoning traces also uncovered severe interaction friction during queries involving alphanumeric codes (e.g., *"Quanti ingredienti servono per l'articolo 00700?"*). The model repeatedly fell into the **"superset trap"**: rather than executing the query, the agent paused the conversation to ask whether the user meant `"00700"` or superset strings present in the catalog (such as `"00700F"` or `"100200700"`).

To ensure operational fluidity, an absolute matching constraint was injected into the system instructions:

> *"Se l'utente fornisce un codice (articolo, materiale, piattaforma o unità di misura), questo deve essere considerato una corrispondenza esatta (EXACT_MATCH). Se il codice inserito è presente nel database, non applicare la logica Superset Trap e non richiedere chiarimenti su codici simili."*

This directive enforces immediate query compilation whenever an unambiguous code identifier is detected, eliminating unnecessary interactive turns.

6.5 Visualization Governance: Multi-Scale Datasets and Tool Delegation

A structural hurdle in conversational Business Intelligence arises when visual interactions intersect with physical supply chain realities: heterogeneous Units of Measure and massive scale disparities.

6.5.1 The Multi-Unit of Measure Aggregation Hazard

Amadori's material planning fact table records component volumes across non-fungible physical metrics: packaging film in meters (`MT`), corrugated cardboard in square meters (`MQ`), thermoformed trays in pieces (`NR`), and seasonings in kilograms (`KG`).

Executing an unconstrained aggregate summation ($\sum$ mat_qta) across mixed units produces a mathematically meaningless scalar. Furthermore, instructing an LLM chat agent to dynamically generate "separate native visual charts for each

unit" creates an architectural conflict: the agent compiles a single runtime SQL query and, lacking iterative rendering passes within a single conversational turn, frequently renders duplicated, overlapping, or empty charts.

To enforce analytical rigor, the system configuration implements a two-tier visualization governance policy:

1. **Tabular Segregation as Baseline:** When multiple UoMs are present, the agent is directed to prioritize structured markdown tables where each material row explicitly details its respective `um_cod`, preventing visual aggregation errors.

2. **Advanced Analytics Python Delegation:** For graphical requests across disparate scales, execution is delegated to the integrated Python Advanced Analytics sub-agent.

6.5.2 Scale-Disparity Governance and Logarithmic Visualizations

In food manufacturing, packaging items and bulk raw materials exhibit extreme numerical variances. For instance, the volume of plastic film required for an automated packaging line may exceed several million meters, while an essential micro-ingredient (such as spice extracts) for the identical batch requires only 50 kilograms.

When plotted on a standard linear cartesian chart, the primary component consumes $> 80\%$ of the axis space, flattening secondary components into invisible baselines. To eliminate this issue, the agent's visualization directives enforce quantitative distribution thresholds:

- **Disproportion Assessment:** When a single category exceeds 70% of the aggregate total, or the maximum value exceeds the minimum value by more than a factor of ten ($V_{\max}/V_{\min} > 10$), the agent suppresses unified linear bar charts.

- **Automated Logarithmic Transformation:** The agent routes the dataset to the Python sub-agent, which dynamically configures a logarithmic scale on the dependent axis (`plt.yscale('log')`) or separates the data into two decoupled sub-plots (one for primary macro-volumes, one for trace ingredients).

6.5.3 Contextual Dashboard Deep-Linking: Architectural Intent vs. Sandbox Constraints

To support operational planners requiring comprehensive, multi-tile graphical dashboards (such as the segregated `fabbisogni_um` LookML dashboard described in **??**), an architectural pattern was designed: delegating multi-metric visualizations to dedicated dashboard assets via dynamic URL deep-links pre-populated with runtime contextual filters (e.g., `?filtro_citta=S01&filtro_periodo=next+month`).

However, during empirical enterprise testing, a security sandbox limitation was encountered. Because the Looker Conversational Analytics interface operates within a hardened, iframe-isolated web component, its internal client-side Markdown sanitizer aggressively strips relative hyperlinks and javascript-driven anchors to prevent Cross-Site Scripting (XSS) and interface escape vulnerabilities. Consequently, dynamic links generated by the LLM were rendered as plain, unclickable text strings, forcing users to manually copy and paste target URLs into separate browser tabs.

While this pattern demonstrates clear theoretical value for hybrid conversational-visual architectures, its operational deployment remains constrained by current UI sandboxing implementations. Consequently, in the current production release, the agent relies primarily on native Looker visualization tiles and direct Python analytical renderings, while contextual dashboard deep-linking remains an architectural blueprint targeted for native resolution in future platform updates.

6.6 Validation Framework: The Golden Queries Suite

To evaluate the agent's analytical reliability, conversational resilience, and adherence to enterprise governance, an empirical validation suite of **Golden Queries** was constructed. In enterprise agentic systems, a Golden Query represents an authoritative benchmark pairing: an authentic, often ambiguous business intent expressed in natural language, mapped against its certified semantic resolution, generated SQL query, and expected output rendering.

6.6.1 Benchmarking Criteria and Taxonomy

Each Golden Query is assessed across four structural dimensions:

1. **Intent and Entity Parsing:** Accurate resolution of business concepts, synonyms, and alphanumeric identifiers to certified LookML dimensions.

2. **Deterministic Guardrail Injection:** Verification that mandatory enterprise filters (default platform sets, temporal bounding, `EXACT_MATCH` rules) are applied without user intervention.

3. **Compilation and Syntactic Integrity:** Generation of clean, optimized SQL utilizing correct relational join paths and Symmetric Aggregates without runtime errors.

4. **Output and Visualization Governance:** Selection of the appropriate presentation modality (tabular, native chart, or logarithmic Python visual) with mandatory UoM pairing and proactive follow-up recommendations.

Table 6.1 summarizes the benchmark evaluation matrix across four representative operational queries designed to stress-test distinct failure modes.

ID	Business Prompt	Semantic Mapping	Enforced Guardrails	Output Modality
GQ-1	*"Quale è il fabbisogno lordo diviso tra ingredienti e materiali per i prossimi 3 mesi?"*	`fabbisogno_materiale`, `tipo_materiale`, `um_cod`	Rolling 90-day window; default platforms (`S01--S50`); no Superset Trap.	Segmented Markdown table with UoM pairing; proactive drill-down suggestion.
GQ-2	*"Quale è il fabbisogno del prossimo mese a Cesena?"*	`fabbisogno_materiale`, `piattaforma`, `data_previsione`	Resolve "Cesena" to platform `S01`; bound date to end of next month.	Summary KPI card and aggregated component table.
GQ-3	*"Di quanto I02165 ho bisogno per gli Elaborati a Cesena nei prossimi 10 giorni?"*	`fabbisogno_materiale`, `materiale_lcod`, `lin_des` (Elaborati)	`EXACT_MATCH` on `I02165`; platform `S01`; rolling 10-day filter.	Single-value scalar with unit (`NR`) and supplier lead-time note.
GQ-4	*"Di cosa ho picchi di fabbisogno nei prossimi 3 mesi a Teramo?"*	`fabbisogno_materiale`, `materiale`, `data_previsione`	Platform `S04` (Teramo); statistical variance evaluation.	Python Advanced Analytics chart utilizing `plt.yscale('log')`.

Table 6.1: Validation Benchmark Matrix of Enterprise Golden Queries

6.6.2 Deep Diagnostic Breakdown of Golden Queries

GQ-1: Broad Structural Demand over Medium Horizon

Natural Language Input: *"Quale è il fabbisogno lordo diviso tra ingredienti e materiali per i prossimi 3 mesi?"*

- **Diagnostic Trace Analysis (*Show Thinking*):** The model identifies the primary metric as `fabbisogno_materiale`. It recognizes "ingredienti e materiali" as a request for the `tipo_materiale` dimension. In earlier development phases, this prompt triggered the Python interpreter; in the target release, the agent immediately maps the field to the backend dimension `l_art_jde.tipo_materiale`. Because the user specifies "prossimi 3 mesi", the agent overrides the standard one-month default and applies a relative date filter: `data_previsione_date IN (CURRENT_DATE() to CURRENT_DATE() + 90)`.
- **Generated LookML Filter State:**

```
explore: material_planning
fields: [materiale.tipo_materiale,
   f_material_planning.um_cod, f_material_planning.
   fabbisogno_materiale]
filters:
  f_material_planning.data_previsione_date: "90 days
     "
  l_ptf.piattaforma_cod: "S01,S02,S04,S05,S10,S50"
```

- **Output Governance:** Because the query spans multiple UoMs (`KG` for ingredients, `NR`/`MT` for packaging), the agent refrains from compiling a misleading aggregate chart. It outputs a formatted markdown table grouped by `tipo_materiale` and `um_cod`, appending two follow-up prompts regarding which specific item family drives packaging requirements.

GQ-2: Geographic Resolution and Temporal Fallbacks

Natural Language Input: *"Quale è il fabbisogno del prossimo mese a Cesena?"*

- **Diagnostic Trace Analysis (*Show Thinking*):** The input contains an informal geographic entity ("Cesena") without an explicit warehouse or plant identifier. The agent inspects the synonyms mapped to `l_ptf.piattaforma` and correlates Cesena with operational platform code `"S01"` (Stabilimento

Cesena). The temporal phrase "prossimo mese" is parsed into an exact calendar window from the first to the final day of the subsequent month.

- **Generated SQL Formulation (Compiled Excerpt):**

```
SELECT
    l_ptf.PTF_DES AS piattaforma_des,
    l_um.UM_COD AS um_cod,
    SUM(f_material_planning.MAT_QTA) AS
       fabbisogno_materiale
FROM material_planning_export.F_MATERIAL_PLANNING AS
    f_material_planning
JOIN material_planning_export.L_PTF AS l_ptf
    ON f_material_planning.PTF_ID = l_ptf.PTF_ID
JOIN material_planning_export.L_UM AS l_um
    ON f_material_planning.MAT_UM_ID = l_um.UM_ID
WHERE l_ptf.PTF_COD = 'S01'
  AND f_material_planning.DATA_MP BETWEEN
      DATE_TRUNC(DATE_ADD(CURRENT_DATE(), INTERVAL 1
          MONTH), MONTH)
      AND LAST_DAY(DATE_ADD(CURRENT_DATE(), INTERVAL
          1 MONTH))
GROUP BY 1, 2
ORDER BY 3 DESC;
```

- **Output Governance:** The response returns aggregated requirements by UoM for Cesena, followed by a proactive prompt offering to break down the total by commercial finished product line (`l_lin`).

GQ-3: Exact Item Code and Line Filtering without Superset Trap

Natural Language Input: *"Di quanto I02165 ho bisogno per gli Elaborati a Cesena nei prossimi 10 giorni?"*

- **Diagnostic Trace Analysis (*Show Thinking*):** This query stresses code disambiguation. The token `"I02165"` is identified as a raw material alphanumeric code. Under previous unconstrained configurations, the agent hesitated, querying the user whether it should match substrings. Under the active `EXACT_MATCH` instruction, it binds `I02165` directly to `materiale.articolo_lcod`. The token "Elaborati" is matched via synonyms to the product line dimension (`l_lin.lin_des = 'Elaborati'`). The geographic filter sets platform `S01`, and the timeframe sets a rolling 10-day window.

- **Output Governance:** The query targets a single homogeneous component (thermoformed trays). Since only one UoM exists (`NR`), the agent generates a prominent single-value metric card alongside a daily consumption line chart. It concludes by suggesting an inventory stock check against supplier delivery schedules.

GQ-4: Anomaly Detection and Visual Skew Handling via Python

Natural Language Input: *"Di cosa ho picchi di fabbisogno nei prossimi 3 mesi a Teramo?"*

- **Diagnostic Trace Analysis (*Show Thinking*):** The intent demands anomaly and peak detection ("picchi di fabbisogno"), which exceeds native SQL aggregation. The agent maps "Teramo" to platform `S04`, bounds the date to 90 days, retrieves weekly material time-series data, and recognizes an extreme scale disparity: primary packaging film volumes exceed 10^6 meters, whereas ingredient additives remain below 10^2 kilograms.
- **Delegation to Advanced Analytics Python Sub-Agent:** Recognizing that $V_{\max}/V_{\min} > 10^4$, native Looker bar charts are suppressed. The agent spawns the Python sub-agent to compute rolling standard deviations and render a logarithmic time-series plot:

```python
import matplotlib.pyplot as plt

# Filter and calculate weekly requirement variance
df['rolling_avg'] = df.groupby('materiale')['fabbisogno'].transform(lambda x: x.rolling(2, min_periods=1).mean())
df['spike'] = df['fabbisogno'] > (df['rolling_avg'] * 1.5)

# Scale Disparity Handling: Logarithmic projection
plt.figure(figsize=(10, 5))
for mat, group in df.groupby('materiale'):
    plt.plot(group['settimana'], group['fabbisogno'], label=mat)

plt.yscale('log')
plt.title("Rilevazione Picchi di Fabbisogno - Teramo (Scala Logaritmica)")
plt.ylabel("Fabbisogno (Scala Logaritmica)")
```

```
plt.xlabel("Settimana di Pianificazione")
plt.grid(True, which="both", ls="--")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
```

- **Output Governance:** The model returns the customized logarithmic plot highlighting two specific materials showing statistically abnormal spikes in weeks 48 and 50, followed by actionable suggestions to investigate planned marketing promotions for those weeks.

6.7 Socio-Technical Impact: Transforming the Role of the Data Office

The deployment of Conversational Analytics powered by Gemini in Looker achieves far more than an incremental improvement in interface usability: it initiates a structural transformation in the socio-technical dynamics of enterprise data management.

Historically, Amadori's Data Office operated primarily as an operational reporting service desk. Analysts spent substantial portions of their working hours fielding ad-hoc data requests, writing custom SQL extracts, diagnosing report discrepancies, and building fragmented dashboards for individual departmental users. This operating model created operational friction, sustained data silos, and left little capacity for strategic initiatives.

By introducing the governed Semantic Layer and the agentic interface, the division of labor across the organization is fundamentally reconfigured:

- **Democratization with Guardrails for Operational Planners:** Business users, supported by embedded **Data & AI Champions**, obtain immediate self-service access to operational intelligence. Planners can query data using natural business vocabulary, confident that enterprise constraints (platform perimeters, exact code matching, and unit segregation) are deterministically enforced without requiring manual query construction.

- **Evolution of the Data Office into a Governance Center:** Rather than producing disposable reports, the Data Office ascends to an architectural steward. Its core responsibilities shift toward defining, curating, and certifying reusable data assets: enriching the Data Catalog, standardizing LookML semantic entities, certifying upstream ELT pipelines, and monitoring LLM diagnostic logs to identify emerging business vocabulary.

Through this transformation, the gap between operational data generation and analytical data consumption identified in Chapter 1 is systematically closed. The Conversational Data Agent becomes a reliable cognitive bridge, grounded in certified enterprise semantics and capable of supporting complex supply chain decision-making at enterprise scale.

Bibliography

[1] Christine L. Borgman. *Big Data, Little Data, No Data: Scholarship in the Networked World*. MIT Press, Cambridge, MA, 2015.

[2] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999.

[3] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, CA, 1970.

[4] Danah Boyd and Kate Crawford. Critical questions for big data: Provocations for a cultural, technological, and scholarly phenomenon. *Information, Communication & Society*, 15(5):662–679, 2012.

[5] DAMA International. *The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK2 Revised)*. DAMA International, revised edition, 2024.

[6] Thomas H. Davenport and Jeanne G. Harris. *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press, Boston, MA, updated edition, 2017.

[7] Zhamak Dehghani. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, Sebastopol, CA, 2022.

[8] Darren Edge, Ha Trinh, Aidan Newman, Joshua Christopher, Andrew Evans, Shenghao Wang, Kevin Larson, Robert Foster, Kyle Bennett, Scott Jones, et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[9] Google Cloud. Best practices for configuring conversational analytics and ai agents in looker, 2024. Accessed: 2026-10-15.

[10] Google Cloud. Lookml and explore best practices for high-performance queries, 2024. Accessed: 2026-10-15.

[11] Google Cloud. Create and manage explore data agents in looker conversational analytics, 2026. Accessed: 2026-10-15.

[12] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[13] Joseph Hilger, Lulit Tesfaye, and Zachary Wahl. *Bridging Knowledge, Data, and AI: Harnessing the Semantic Layer Framework to Drive Intelligence*. Professional and Applied Computing Series. Springer Nature, Cham, Switzerland, 1st edition, 2026.

[14] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, José Emilio Labra Gayo, Sabrina Kirrane, Sebastian Neumaier, Axel Polleres, Roberto Navigli, Axel-Cyrille Ngonga Ngomo, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[15] Rob Kitchin. *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences*. SAGE Publications, London, 2014.

[16] Sabina Leonelli. *Data-Centric Biology: A Philosophical Study*. University of Chicago Press, Chicago, IL, 11 2016.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[18] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, NY, 1995.

[19] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, Stanford, CA, 2001.

[20] Dip Bharatbhai Patel. Comparative analysis of data visualization tools: Tableau, power bi & looker. *The Eastasouth Journal of Information System and Computer Science*, 1(1):55–59, 2023.

[21] Pranitha Potturi. Demystifying semantic layers in business intelligence platforms. *Sarcouncil Journal of Engineering and Computer Sciences*, 4(8):612–618, 2025.

[22] James Richardson, Kurt Schlegel, Rita Sallam, Austin Kronz, and Julian Sun. Magic quadrant for analytics and business intelligence platforms. Technical Report G00467317, Gartner, February 2021.